

コードの時系列変化を考慮した 機械学習に基づく欠陥混入の低減手法

鈴木研究室 2410064

笹川 尋翔

1.1 背景: ソフトウェア開発と欠陥リスク

- CI/CDの普及によりリリース後もソースコードの継続的かつ頻繁な変更が不可欠となった
- 欠陥の発見が遅れるほど、修正に必要な工数とコストは指数関数的に増大する
- 76%の開発者が、変更作業による意図しないバグ混入のリスクを認識している^[1]

[1] M. Kim et al., "An empirical study of refactoring challenges and benefits at microsoft," 2014'

1.2 研究の目的: 意思決定支援モデルの構築

- 特徴量の時間的变化を通じてバグ混入リスクを特定し、限られたレビュー労力を効率的に配分する意思決定支援モデルを提示する
 1. 構成要素の統合: メソッド単位の局所的変化とコミット単位の全体的変化を組み合わせる
 2. 不規則な時系列の考慮: コミットを点過程データとして扱い、直前状態からの差分を学習する
 3. レビュー労力の推定: 規模と複雑さに着目した労力指標を定義し、発見率を向上させる

1.3 関連研究と既存手法の課題

- 循環的複雑度^[2]やコード行数などは静的な状態を示すが、開発プロセスの動的な特徴を反映できない
- 変更の規模や頻度が強い予測因子となることが示されているが^[3]、時系列構造を明示的に考慮した手法は少ない
- 20%の労力で35%の欠陥特定が可能という報告があるが^[4]、労力の推定精度に課題が残る

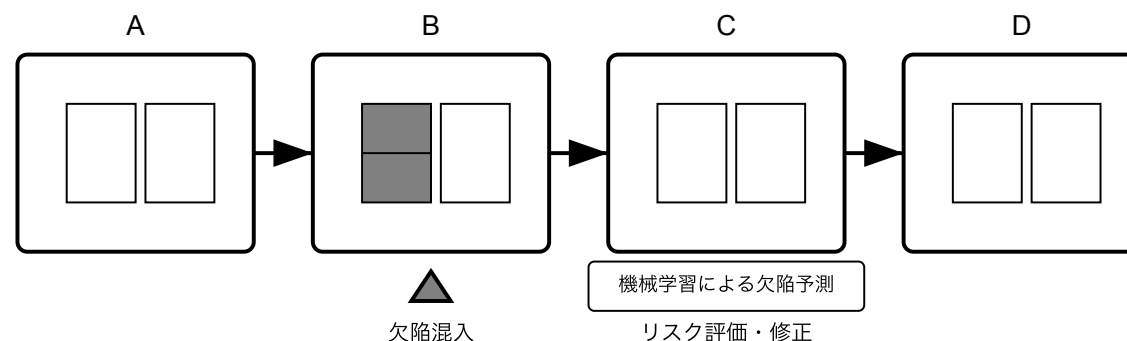
[2] T. J. McCabe, "A complexity measure," 1976

[3] T. L. Graves et al., "Predicting fault incidence using software change history," 2000

[4] Y. Kamei et al., "A large-scale empirical study of just-in-time quality assurance," 2013'

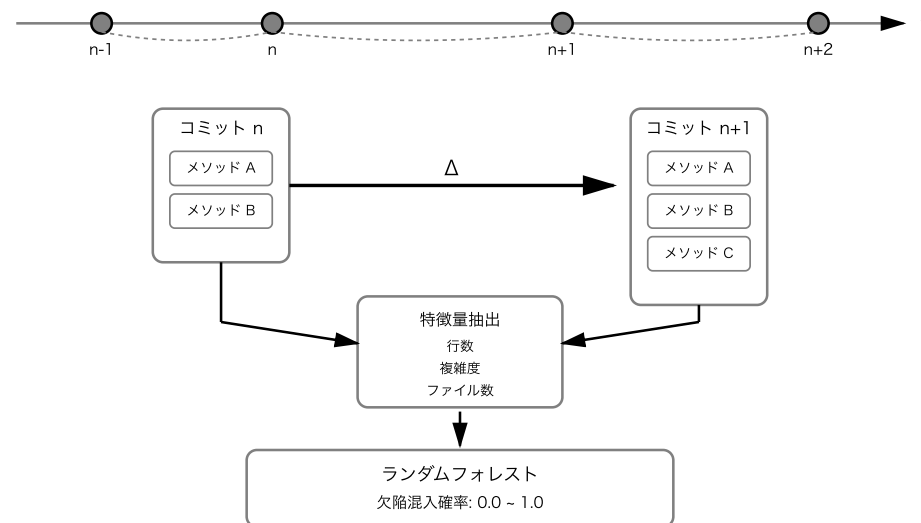
2.1 提案手法1: 不規則な時系列情報の活用

- コミットを不規則に発生するイベントと定義し、発生タイミングが持つリスク情報を活用する
- 絶対値ではなく、直前の状態からの変化量に着目する
 - 例：現在の行数ではなく何行増えたか、構造がどれほど複雑化したかを重視
- 短期間での急激な複雑化など、静的な解析では捉えきれない欠陥の予兆を識別可能にする



2.2 提案手法2: 複数要素の変更メトリクスの活用

- 局所的視点（メソッド単位）：コード行数、トークン数、循環的複雑度の各変化量を用いる
- 全体的視点（コミット単位）：変更ファイル数、追加・削除行数、およびエントロピーを用いた変更の分散度を用いる



2.3 特徴量設計: メソッド識別子の処理

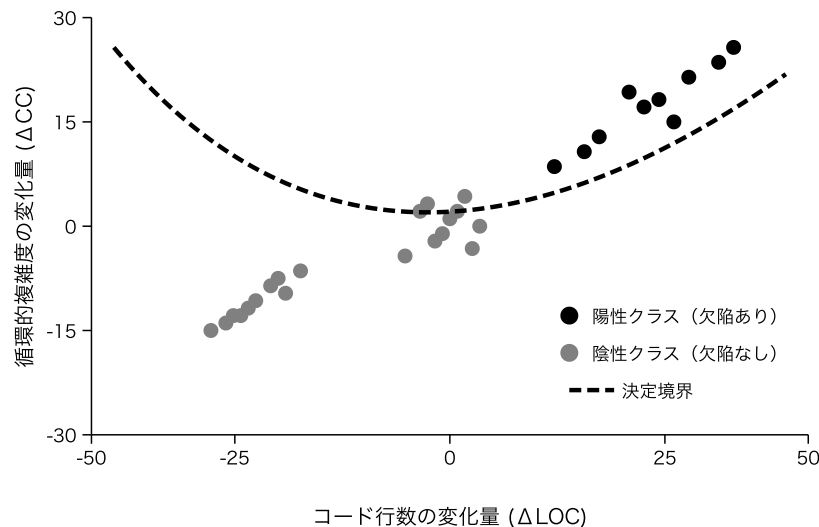
- 機能を表す識別子は欠陥発生率と関連が深いいため、これを特徴量化する
- キャメルケース等に基づき分解し、単語レベルでの類似性を認識可能に
 - 例: getUsername は user と name に分解
- TF-IDFによるベクトル化: 頻出語を抑制しつつ、特定の機能を示す重要なキーワードに重みを付与し、汎化性能を高める

2.4 労力を考慮したレビュー優先度付け

- 補正済みレビュー労力: 変更行数と広がりを経統合し、対数変換により巨大な変更の影響を緩和した指標を定義する
 - $W_i = \log_2(C_i \times N_i^{\overline{H}_i} + 1)$
- 組み合わせ最適化: 少ない労力で高い欠陥発見率を達成できるレビュー対象の選定
- 貪欲法による求解: 単位労力あたりの欠陥混入確率が高い順に選定する

3.1 実験環境と対象プロジェクト

- 欠陥混入から修正までの対応関係を持つBugHunterデータセット[5]
- 非線形関係の抽出と解釈性に優れたランダムフォレストを使用し、F1スコアとAUCによる評価、McNemar検定による有意性検定を行う



[5] R. Ferenc et al., "An automatically created novel bug dataset and its validation in bug prediction," 2020'

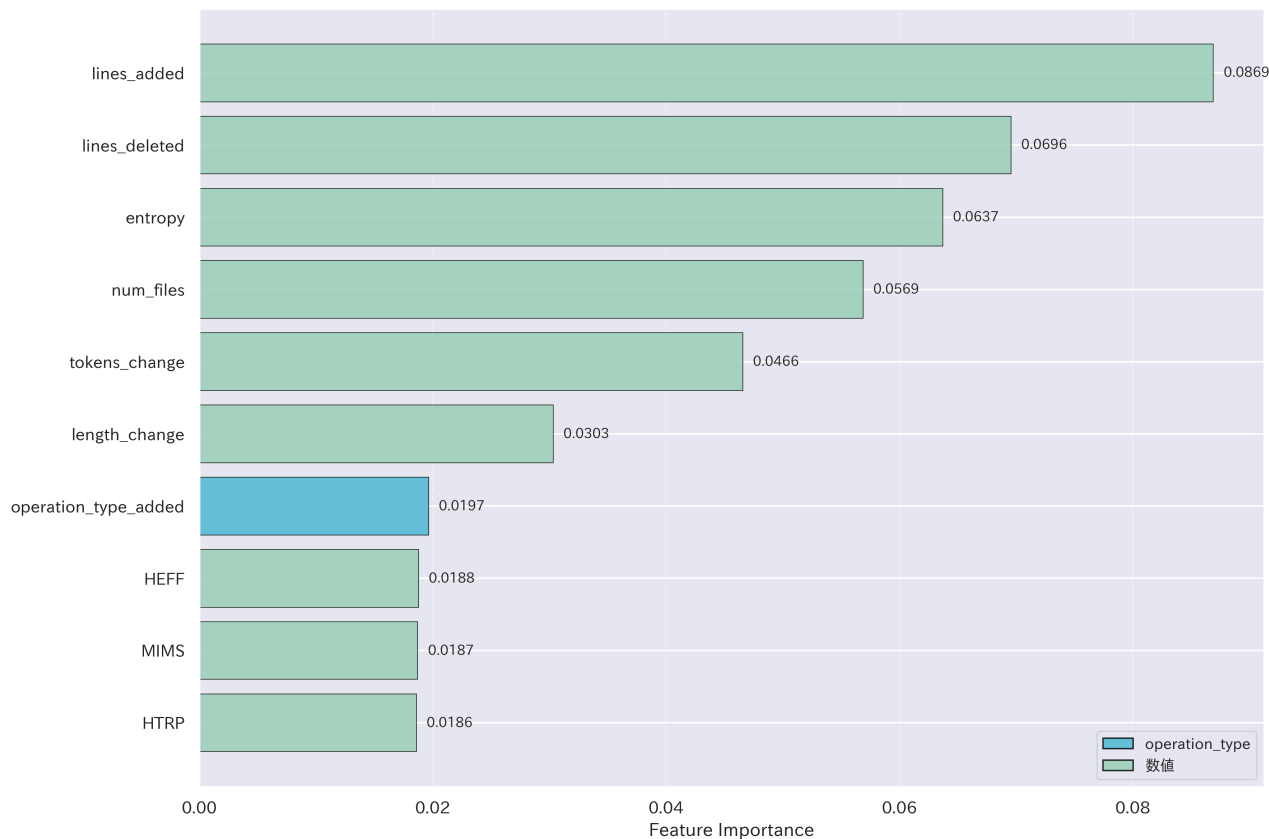
3.2 実験結果1: 予測性能の向上

- 全てのプロジェクトで性能が向上し、F1スコアの平均改善幅は0.21を達成した
- 最終的なAUCは全プロジェクトで0.91を超え、高い識別能力を確認した

プロジェクト	既存手法 (F1)	提案手法 (F1)	改善幅
Elasticsearch	0.575	0.767	+0.192
Hazelcast	0.678	0.790	+0.112
Neo4j	0.478	0.742	+0.264
Netty	0.455	0.747	+0.292
OrientDB	0.483	0.701	+0.218

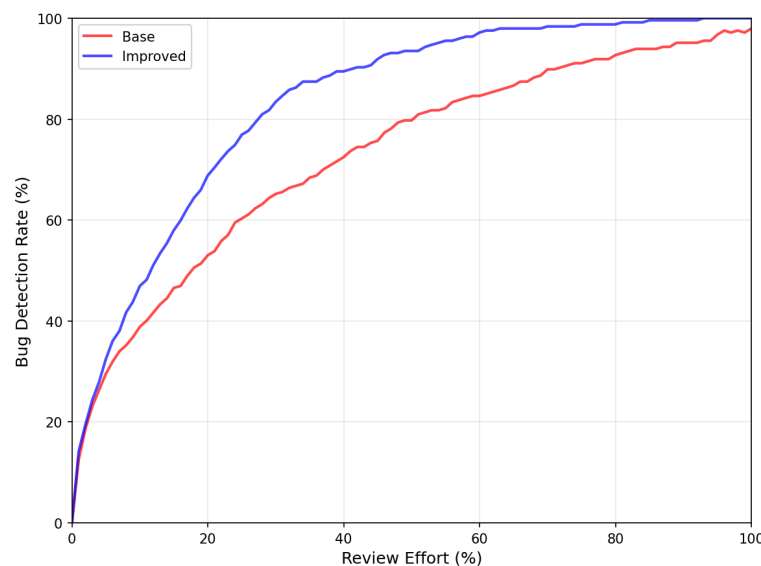
実験結果1: 予測性能の向上

- 特徴量重要度の分析により、コミット単位の追加行数や変更ファイル数が特に予測に寄与することが判明した



3.3 実験結果2: レビュー効率の改善

- 提案手法により、欠陥の70～75%を20%のレビュー労力で特定
- 労力40%時点での欠陥発見率は平均87.0%に達し、既存手法（75.8%）から11.3%改善
- 労力40%時点において、既存手法に対し統計的に有意な改善を確認した



4.1 考察1: プロジェクト特性と不均衡データ

- 構造的メトリクスのみでは予測が困難なプロジェクトほど、時系列変化情報の追加による改善幅が大きくなる
- 陽性クラスの割合が低いほど改善効果が顕著であり、変化の特徴が少数派の識別に有効に機能している
- 複数のドメイン（検索、DB、通信等）に対し、一貫して高い性能を発揮した

4.2 考察2: 変更規模と欠陥確率の逆転現象

- PDP分析により、変更規模が小さいほど欠陥混入確率が高いという傾向が観察された
- 要因の考察:
 1. 小規模な変更は部分的なバグ修正であることが多く、新たな不具合を誘発しやすい^[6]
 2. 小規模な変更はレビューが容易なため、潜在的な欠陥が発見されやすく、記録に残る可能性が高い

[6] Abram Hindle et al., "What do large commits tell us? a taxonomical study of large commits," 2008

4.3 評価と手法の制約

- 欠陥の特定に使用したSZZアルゴリズムには、特定の修正を見落とすなどの課題が存在する^[7]
- 本研究は特定の言語（Java）のプロジェクトを対象としており、異なる言語特性への適用には再検証が必要である
- 予測精度の最大化だけでなく、開発者がなぜ予測されたかを理解できる透明性を重視して構築した

[7] S. Herbold et al., "Problems with szz and features," 2022

5.1 結論

- 複数の構成要素のメトリクスを活用することで、欠陥によるメトリクスの変化に敏感な欠陥モデルを構築した
- コミット間の不規則な変化を捉えることで、従来の静的な解析を上回る精度での予測を可能にした
- 労力を考慮したレビュー優先度付けにより、現実の開発環境において品質保証を効率化するモデルを提示した

5.2 今後の展望: 変更目的の推定と予防

- 変更規模だけでなく、機能追加やリファクタリングといった変更の目的と欠陥原因を関連付ける必要がある
- コミットメッセージ、コード構造、変更の文脈、開発者の変更履歴などの開発プロセスにかかわる情報を用いてモデルを構築する
- 欠陥が混入した原因を突き止めることで、具体的な予防ガイドラインの提示を目指す